

Technical Assignment

Assignment Title: Weekly Report Generator & Team Dashboard

Objective

Build a full-stack web application that allows individual team members to submit structured weekly work reports and allows managers to view and analyze those reports across the whole team through a consolidated dashboard.

This assignment evaluates your ability to design multi-user systems with role-based access, implement clean UI components, structure backend services, integrate AI-assisted features, and manage data efficiently.

You are free to choose your preferred stack, but modern frameworks such as React, Next.js, or similar frontend frameworks are preferred.

Core Requirements

1. User Authentication & Roles

Implement authentication with role-based access.

Required roles:

- Team Member — can create and manage their own weekly reports
- Manager / Admin — can view and analyze reports across all team members

Required features:

- User registration
- Login / Logout
- Password protection
- Secure session handling
- Role assignment (e.g. by an admin, or at signup)

2. Personal Weekly Report Page

Every user must have their own dedicated page for creating and managing their weekly reports.

The report structure must be fixed and identical for every user — the same set of fields, in the same order, across the whole team. Users should not be able to customize, reorder, or add their own fields. This ensures reports stay consistent and comparable across the team on the manager's dashboard.

Each report should contain:

- Week / date range
- Project or category tag
- Tasks completed
- Tasks planned for next week

- Blockers / challenges
- Hours worked (optional)
- Optional notes or links

Features required:

- Create weekly report
- Edit report (before/after submission, as per your design)
- Submit report
- View own report history, organized by week

3. Team Dashboard (Manager View)

Managers should be able to view and analyze reports submitted by the whole team.

Features:

- View all team members' reports for a selected week
- Filter reports by team member
- Filter reports by project / category
- Filter reports by date range
- Track submission status (submitted / pending / late) per team member

4. Projects / Categories

Allow projects or work categories to be managed and attached to report entries.

Examples: Client A, Internal Tooling, R&D, Marketing.

Features:

- Add project / category
- Edit project / category
- Delete project / category
- Assign team members to relevant projects (optional)

5. Dashboard & Visual Insights

A data-driven dashboard must be implemented for managers.

Dashboard should include:

Summary metrics:

- Total reports submitted this week
- Submission compliance rate (submitted vs pending)
- Number of open blockers across the team

Visual insights — include charts such as:

- Tasks completed trend over time (per person or team-wide)
- Report submission status by team member
- Workload / task distribution by project
- Recent reports / activity feed

Candidates may use any charting library. Examples: Recharts, Chart.js, E-Charts.

6. AI Chat Assistant [Good to Have]

As an optional enhancement, candidates may integrate an AI-powered chat assistant into the application. This is not mandatory but will be viewed favorably during evaluation.

Suggested capabilities:

- Conversational Q&A for managers about team activity (e.g. "What did the design team work on last week?")
- AI-generated team summary highlighting completed work, recurring blockers, and workload imbalances
- Simple in-app chat widget UI for interacting with the assistant

Candidates may use any LLM provider (e.g. Anthropic Claude, OpenAI, or an open-source model) and any integration approach (direct API calls, function calling / tool use, or a lightweight RAG setup over stored reports). Please document your approach, prompt design, and any data-privacy considerations in the presentation if this feature is implemented.

Technical Requirements

Frontend

Preferred frameworks:

- React
- Next.js
- Angular
- Vue

UI expectations:

- Responsive layout
- Component-based structure
- Clean reusable components
- Well-structured folder organization
- Clear separation between the personal report page and the team dashboard

Backend

You may use any backend framework such as:

- Node.js / Express
- Python / FastAPI
- NestJS
- Django
- Spring Boot

Required:

- REST API
- Proper request validation
- Role-based access control on relevant endpoints
- Clean service / controller structure

Database

You may choose any database. Examples:

- PostgreSQL
- MySQL
- MongoDB

Deliverables

Candidates must submit the following:

1. Presentation

Candidates must prepare a short technical presentation explaining their solution. The presentation should cover:

- System architecture overview
- Database design explanation
- Key frontend components (personal page vs. team dashboard)
- API design and backend structure, including role-based access
- AI Chat Assistant approach, if implemented (Good to Have)
- Challenges faced and how they were solved
- Possible future improvements

Accepted formats: Google Slides.

2. GitHub Repository

The repository must contain:

- Frontend code
- Backend code
- Setup Instructions README file covering: 1) Installing dependencies, 2) Running frontend, 3) Running backend, 4) Running database

3. ER Diagram

Provide an Entity Relationship Diagram that explains your database design, including how users, roles, projects, and reports relate to one another.

Accepted formats: Diagram link.

4. Video Explanation / Demo

If the application is fully or partially developed, candidates must submit a short video explanation (demo) of the application. The video should ideally cover:

- Application walkthrough (as both a Team Member and a Manager, if roles are implemented)
- Key features implemented
- Frontend flow
- AI Chat Assistant demo, if implemented (Good to Have)

Accepted formats: Google Drive link.

Evaluation Criteria

Your solution will be evaluated based on:

- System design
- Code quality
- UI structure
- Component reusability
- API design, including role-based access control
- Database design
- Documentation clarity
- Technical explanation during the presentation
- *AI Chat Assistant implementation (bonus points, not mandatory)*

Submission

Please share:

- GitHub repository link
- Video link
- ER diagram link
- Presentation file link